

Dossier technique — AEGIS Mini-SOC

Spécification complète, protocole de mesure et résultats détaillés

Mohamed Aziz BAOUEB • <https://github.com/aziz3r/aegis-mini-soc>

Ce document accompagne le rapport de projet. Il rassemble la spécification exhaustive du système — caractéristiques, modèles, API, modèle de données, inventaire des tests — ainsi que les résultats complets, famille par famille, et le détail du protocole qui les produit.

Table des matières

Partie I — Spécification du système	3
1 Vue générale	3
1.1 Nature et périmètre	3
1.2 Chaîne de traitement	3
2 Assemblage en flux	4
2.1 Définition de l'unité d'analyse	4
2.2 Déclencheurs d'export	4
2.3 Mécanisme d'expiration	4
2.4 Horloge	5
3 Les 52 caractéristiques	5
3.1 Caractéristiques intrinsèques au flux (33)	5
3.2 Comportement de l'hôte source (15)	5
3.3 Comportement de l'hôte destination (4)	5
3.4 Une absence volontaire	5
3.5 Garantie d'ordre	5
4 Le modèle à deux étages	6
4.1 Étage A — détection d'anomalie non supervisée	6
4.2 Calibration par fonction de répartition empirique	6
4.3 Étage B — classification de la famille	6
4.4 La cascade de précision	6
4.5 Attribution par occlusion	7
5 Le seuil adaptatif	7
5.1 Pourquoi la règle n médiane + k MAD z échoue ici	7
5.2 Les deux garde-fous	8
6 Corrélation et triage	8
6.1 Calcul de la priorité	8
6.2 Cycle de vie d'un incident	9
Partie II — Mesures	10
7 Protocole	10

8 Résultats globaux	10
9 Résultats par famille	10
9.1 Lecture des deux colonnes	11
9.2 Le cas de la balise	11
10 Précision : deux niveaux de lecture	11
Partie III — Mise en œuvre	12
11 Interface et API	12
11.1 Les huit écrans	12
11.2 Transport et authentification	12
11.3 Contre-pression	12
12 Modèle de données	12
13 Les trois sources de trafic	13
13.1 Réseau de laboratoire	13
13.2 Rejeu de capture	13
13.3 Capture réelle	13
14 Inventaire des tests	13
15 Déploiement	14
16 Limites et orientations	14

Partie I — Spécification du système

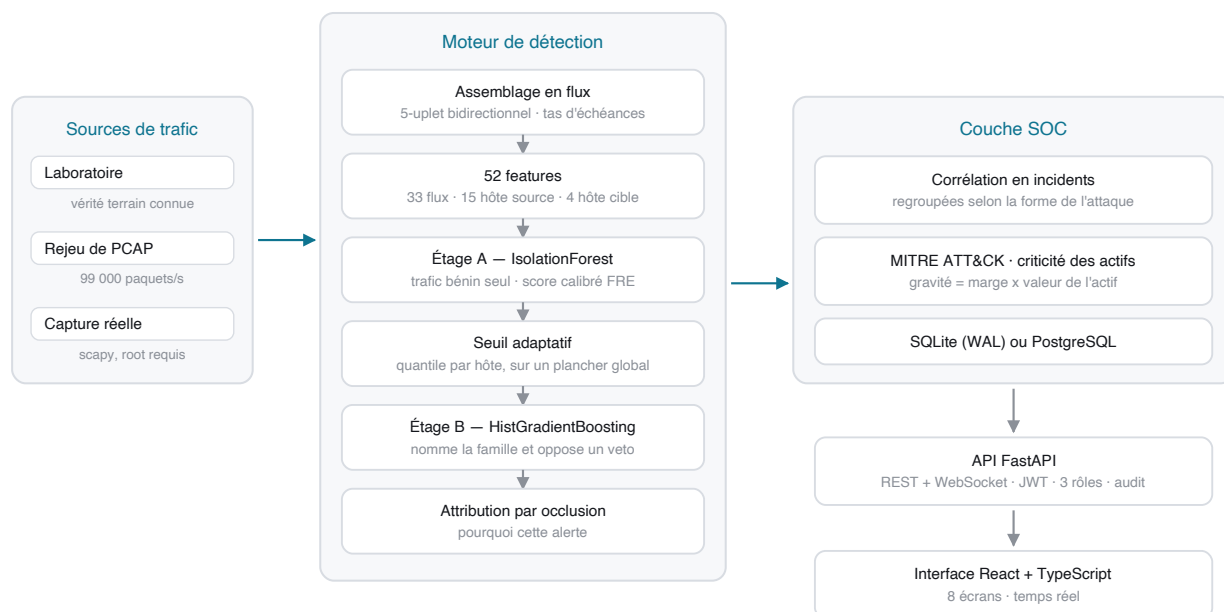
1 Vue générale

1.1 Nature et périmètre

AEGIS est un système de détection d'intrusion réseau (NIDS) **comportemental**, accompagné d'une interface de supervision et de triage. Il n'utilise aucune base de signatures : il apprend la distribution du trafic normal et signale les écarts.

Nature	Détection d'intrusion réseau comportementale + poste d'analyste
Langages	Python 3.11+ (moteur, API) ; TypeScript 5 strict (interface)
Volume	≈9 800 lignes : 5 800 backend, 3 100 interface, 900 tests
Tests	94, tous passants
Déploiement	mono-processus, sans dépendance système ; PostgreSQL optionnel
Démarrage	<code>./scripts/demo.sh</code>
Licence	MIT

1.2 Chaîne de traitement



Les trois sources traversent exactement la même chaîne : rien du chemin de détection n'est simulé. L'horloge est celle des paquets, jamais celle du mur — c'est ce qui garantit qu'un PCAP rejoué à x60 produit les mêmes flux que la capture d'origine.

L'étage A n'a jamais vu d'attaque : il peut donc en signaler une qu'il ne connaît pas. L'étage B ne décide pas s'il faut alerter, il nomme ce que l'anomalie ressemble — et supprime les fausses alertes qu'il reconnaît comme du trafic normal.

Figure 1 — La chaîne complète, des paquets jusqu'à l'interface.

Trois sources de trafic alimentent indifféremment le même moteur : un réseau de laboratoire synthétique produisant des paquets étiquetés, un fichier PCAP rejoué, ou une interface réseau réelle. Le reste de la chaîne ignore la provenance.

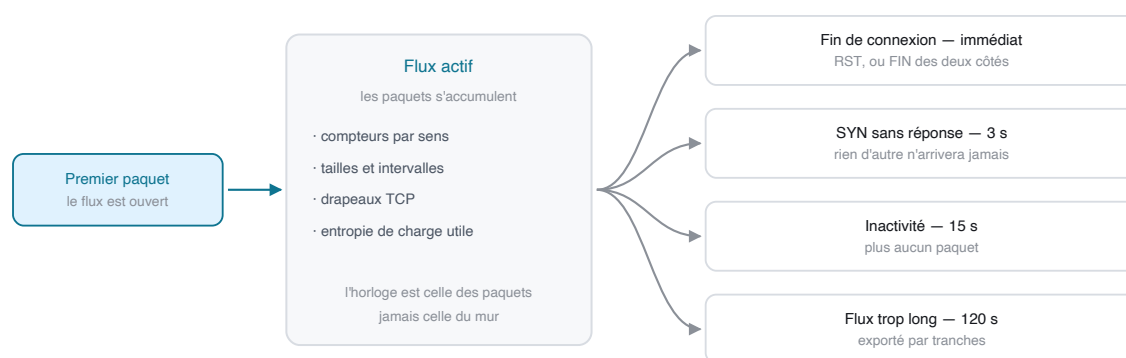
2 Assemblage en flux

2.1 Définition de l'unité d'analyse

L'unité analysée est le **flux bidirectionnel**, identifié par son 5-uplet canonique (adresse source, adresse destination, port source, port destination, protocole), rendu indépendant du sens de manière à ce que $A \rightarrow B$ et $B \rightarrow A$ constituent un seul flux.

2.2 Déclencheurs d'export

Déclencheur	Délai	Justification
Fin de connexion TCP	immédiat	RST observé, ou FIN dans les deux sens
Connexion sans réponse	3 s	un SYN sans réponse est déjà une observation complète
Inactivité	15 s	plus aucun paquet n'est observé sur le flux
Durée d'ouverture	120 s	un flux long est exporté par tranches, pour rester visible



L'export après 3 s d'une tentative de connexion sans réponse n'est pas un détail : un SYN resté sans réponse est déjà une observation complète. Attendre le délai d'inactivité complet ajoutait 15 secondes à la détection de chaque balayage et de chaque inondation — les deux attaques composées presque entièrement de SYN sans réponse. Latence médiane : de 16 s à 3 s.

Figure 2 — Les quatre façons dont un flux quitte la table.

Le cas des connexions sans réponse n'est pas un détail d'optimisation. Un balayage de ports et une inondation SYN sont composés presque entièrement de tentatives sans réponse ; les conserver pendant le délai d'inactivité complet ajoutait quinze secondes à leur détection. Le délai dédié de trois secondes a ramené la latence médiane de détection de seize à trois secondes.

2.3 Mécanique d'expiration

L'expiration repose sur deux tas d'échéances paresseux : un pour l'inactivité, un pour la durée d'ouverture. Chaque paquet empile une échéance ; les entrées périmées — le flux a reçu un paquet depuis, ou a déjà été exporté — sont reconnues au dépilage grâce à un compteur de génération propre à chaque flux, puis rejetées.

Implémentation	Complexité par paquet	Débit mesuré
Parcours de la table des flux actifs	$O(n_{\text{actifs}})$	12 000 paquets/s
Tas d'échéances paresseux	$O(\log n)$ amorti	33 000 paquets/s

L'équivalence stricte entre les deux implémentations est vérifiée par un test, sur quatre minutes de trafic mixte : les flux produits sont identiques en identité, nombre de paquets et volume. Une optimisation non prouvée est un bug qui attend.

2.4 Horloge

Le champ `Packet.ts` porte toujours l’horloge de **capture**, jamais l’heure d’arrivée réelle. Les délais d’expiration s’expriment donc en temps de capture, ce qui garantit qu’un fichier PCAP rejoué à soixante fois la vitesse produit exactement les mêmes flux que la capture d’origine. L’heure d’arrivée est enregistrée séparément, pour l’affichage uniquement.

3 Les 52 caractéristiques

3.1 Caractéristiques intrinsèques au flux (33)

Sous-groupe	Caractéristiques
Volume	durée, paquets aller, paquets retour, paquets totaux, octets aller, octets retour, octets totaux
Débit	paquets par seconde, octets par seconde
Dissymétrie	ratio paquets aller/retour, ratio octets descendant/montant
Tailles	moyenne, écart-type, minimum, maximum
Temporalité	intervalle moyen, écart-type, minimum, maximum
Drapeaux TCP	SYN, ACK, FIN, RST, PSH, URG, poignée de main aboutie
Protocole	TCP, UDP, ICMP (trois indicatrices)
Charge utile	longueur moyenne, entropie de Shannon normalisée, part de paquets sans données
Port	le port de destination est-il un port système (< 1024)

3.2 Comportement de l’hôte source (15)

Mesuré sur deux fenêtres glissantes, dix et soixante secondes : nombre de flux, destinations distinctes, ports distincts, taux de SYN sans établissement, part de connexions échouées, octets sortants et entrants, ratio sortant/entrant, part de destinations jamais contactées auparavant, durée moyenne des flux, répétitions maximales vers une même cible, et **régularité de type balise**.

Cette dernière mérite une définition : elle vaut $1 - CV$, où CV est le coefficient de variation des intervalles entre contacts successifs vers la destination la plus sollicitée. Une valeur proche de 1 traduit une périodicité d’horloge — ce que fait un implant qui contacte son serveur de commande, et ce qu’un humain ne fait jamais.

3.3 Comportement de l’hôte destination (4)

Nombre de flux reçus, sources distinctes, taux de SYN non établis et part de connexions échouées, sur dix secondes.

Ces quatre caractéristiques existent pour un cas précis : une inondation dont les adresses sources sont **usurpées** est invisible côté source, puisque chaque adresse falsifiée n’émet qu’un seul paquet et paraît donc inactive. Seule l’agrégation sur la victime la rend détectable.

3.4 Une absence volontaire

Le numéro de port de destination *brut* n’est pas une caractéristique. Un modèle à arbres le mémoriserait — 22 pour la force brute SSH, 53 pour le tunnel DNS — et afficherait des scores excellents sans aucun rapport avec sa capacité à généraliser sur du trafic réel. Seules les propriétés structurelles du port sont conservées.

3.5 Garantie d’ordre

La liste `FEATURE_NAMES` est l’unique source de vérité pour l’ordre des colonnes. Le modèle enregistre cette liste et **refuse de se charger** si elle a changé : une colonne qui se décale silencieusement est le

pire mode de panne d'un système de ce type, puisqu'il continue de produire des scores d'apparence normale.

4 Le modèle à deux étages

4.1 Étage A — détection d'anomalie non supervisée

Algorithme	IsolationForest, 220 arbres, <code>max_samples = 4096</code>
Normalisation	RobustScaler, quantiles 5–95
Contamination	0,02 — une capture « bénigne » contient toujours un peu d'étrangeté réelle
Entraînement	trafic bénin exclusivement , 22 295 flux
Sortie brute	score non borné, non interprétable

N'ayant jamais vu d'attaque, cet étage peut en signaler une qu'il ne connaît pas. C'est la propriété qui justifie l'approche comportementale, et elle serait perdue si on l'entraînait sur des attaques étiquetées.

4.2 Calibration par fonction de répartition empirique

La sortie brute d'un IsolationForest n'est pas interprétable : elle dépend de l'échelle des données et n'a pas d'unité. Elle est donc passée par la fonction de répartition empirique des scores obtenus sur le **trafic bénin d'entraînement**.

Le score devient alors un percentile : 0,97 signifie « plus inhabituel que 97% du trafic normal connu ». Un seuil posé sur ce nombre est directement lisible comme un budget de faux positifs.

Au-delà du maximum observé en trafic bénin, la fonction ne sature pas : elle tend vers 1 de façon asymptotique, ce qui préserve l'ordre entre deux anomalies toutes deux extrêmes.

Validation : appliquée à une loi normale, la calibration retrouve la fonction de répartition de cette loi à $\pm 0,01$ près ($0 \rightarrow 0,500$; $1 \rightarrow 0,841$; $1,96 \rightarrow 0,975$). C'est un test de la suite.

4.3 Étage B — classification de la famille

Algorithme	HistGradientBoosting, 300 itérations, 31 feuilles, $L_2 = 1,0$
Équilibrage	<code>class_weight="balanced"</code> , arrêt anticipé sur 15% de validation
Entraînement	19 397 flux étiquetés, plafonnés à 2 500 par classe
Sortie	famille et confiance ; « inconnu » en dessous de 0,55 de confiance

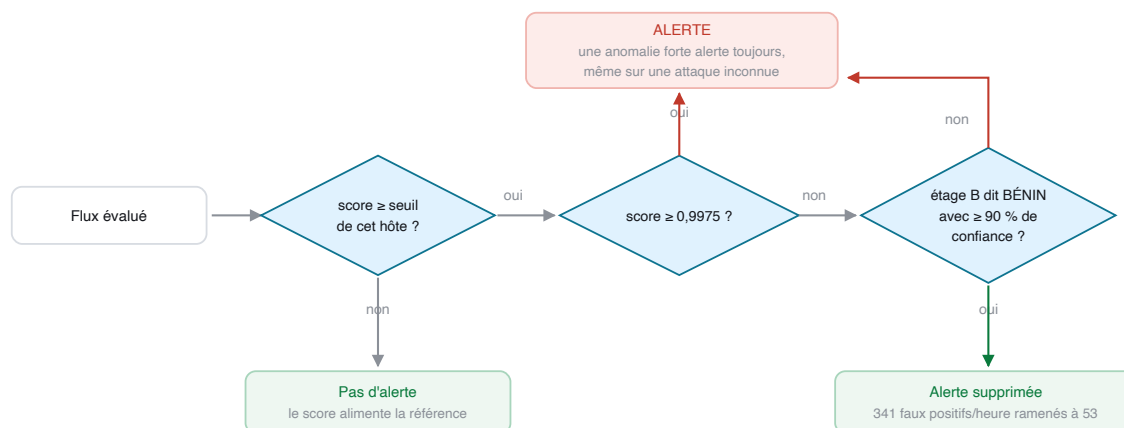
Le plafonnement par classe n'est pas cosmétique : une seule inondation produit des milliers de flux quasi identiques et représenterait 70% du jeu d'entraînement, rendant invisibles les familles rares — balise, exfiltration — qui sont précisément les plus difficiles.

Cet étage ne décide pas s'il faut alerter. Il nomme, et refuse de nommer en dessous du seuil de confiance : une étiquette fausse mais assurée nuit davantage à un analyste qu'une absence d'étiquette.

4.4 La cascade de précision

```
seuil      = seuils.threshold(hote)           # lu AVANT apprentissage
alerte     = score >= seuil
si alerte et famille == BENIGN et confiance >= 0.90 et score < 0.9975 :
    alerte = False                           # veto de l'étage B
seuils.observe(hote, score, alerte)          # les alertes n'alimentent pas la reference
```

La borne de 0,9975 est ce qui rend la cascade acceptable : au-delà, une anomalie forte alerte toujours, quoi qu'en dise le classifieur. Sans elle, une attaque jamais apprise serait étouffée par un modèle qui ne la connaît pas.



L'étage A est réglé pour la sensibilité, ce qui coûte des faux positifs dans la bande juste au-dessus du seuil. Quand le classifieur supervisé reconnaît ce trafic comme normal — et le dit avec assurance — l'alerte est supprimée. Jamais au-delà de 0,9975 : c'est cette borne qui préserve la détection des attaques que le classifieur n'a jamais apprises.

Figure 3 — La décision d'alerter, étage par étage.

	Faux positifs/h	Rappel bruyant	Rappel furtif
Sans cascade	341	99,83%	98,78%
Avec cascade	53	99,82%	98,15%

Table 1 — Effet mesuré de la cascade : six fois moins de faux positifs pour 0,6 point de rappel furtif.

4.5 Attribution par occlusion

Chaque caractéristique est remplacée tour à tour par sa valeur médiane sur le trafic normal, et le flux est réévalué ; la chute du score mesure la part de l'anomalie qui lui est imputable. C'est l'intuition que SHAP formalise, calculée *exactement* pour les sous-ensembles à une caractéristique, en un seul appel vectorisé et sans dépendance supplémentaire.

Le coût est encadré par un budget : une inondation peut lever des centaines d'alertes par seconde, et les expliquer toutes dominerait la boucle sans aucun bénéfice, puisqu'elles sont fusionnées en un seul incident qui n'a besoin que d'une explication. L'incident conserve celle de son alerte la plus forte.

5 Le seuil adaptatif

$$\text{seuil}(\text{hôte}) = \text{borner} \left(\max \left(\text{plancher global}, Q_{0,99}(\text{scores de l'hôte}) \right) \right)$$

Paramètre	Valeur	Signification
Plancher global	0,98	budget de faux positifs, et valeur à froid
Quantile par hôte	0,99	alerter sur le 1% le plus inhabituel de cet hôte
Plafond	0,995	limite jusqu'où un hôte peut relever sa propre barre
Référence minimale	60 flux	en deçà, le plancher s'applique
Fenêtre	512 scores	historique glissant par hôte

5.1 Pourquoi la règle n médiane + k MAD n échoue ici

La première implémentation utilisait cette règle robuste classique. Mesurée, elle a fait tomber le rappel à **14%** — sans qu'aucune métrique de classement ne le signale, puisque le PR-AUC restait à

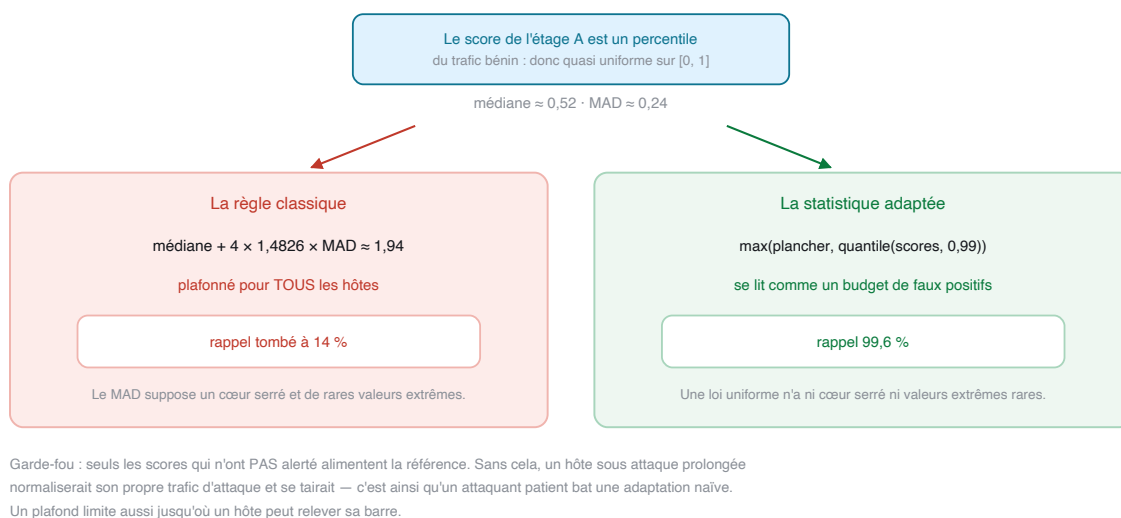


Figure 4 — Pourquoi un quantile et non la règle robuste classique.

0,993.

La cause : les scores de l'étage A étant déjà des percentiles, ils sont quasi uniformes sur le trafic normal. La médiane vaut environ 0,52, l'écart absolu médian 0,24, et $0,52 + 4 \times 1,4826 \times 0,24 \approx 1,94$, valeur plafonnée pour *tous* les hôtes. L'écart absolu médian suppose une distribution à cœur serré avec de rares valeurs extrêmes ; une loi uniforme ne présente ni l'un ni l'autre.

5.2 Les deux garde-fous

Exclusion des scores alertants. Seuls les scores n'ayant pas déclenché d'alerte alimentent la référence. Sans cette règle, un hôte sous attaque prolongée normaliserait progressivement son propre trafic d'attaque et cesserait d'alerter — le mode de défaillance dit « de la grenouille ébouillantée », et la manière dont un attaquant patient défait une adaptation naïve. Un test vérifie que cinq mille scores alertants consécutifs ne déplacent pas le seuil d'un hôte.

Plafond. Un hôte ne peut relever sa barre au-delà de 0,995, afin qu'une attaque lente qui parviendrait malgré tout à s'infiltrer dans une référence ne puisse pas rendre cet hôte insensible.

6 Corrélation et triage

Forme	Familles concernées	Clé
Un vers plusieurs	balayage de ports, balayage réseau	la source
Plusieurs vers un	déni de service, slowloris	la victime
Un vers un	forces brutes, balise, exfiltration, tunnel	la paire

La fenêtre de regroupement est de soixante secondes. Mesuré en fonctionnement : **8 994 alertes fusionnées en un seul incident**.

6.1 Calcul de la priorité

$$\text{priorité} = 100 \times \left(0,50 m + 0,30 \frac{c}{5} + 0,12 f + 0,08 v \right)$$

où m est la marge normalisée au-dessus du seuil *de cet hôte*, c la criticité de l'actif visé (1 à 5), f la confiance du classifieur et v un appoint de volume plafonné. La gravité affichée en découle : critique

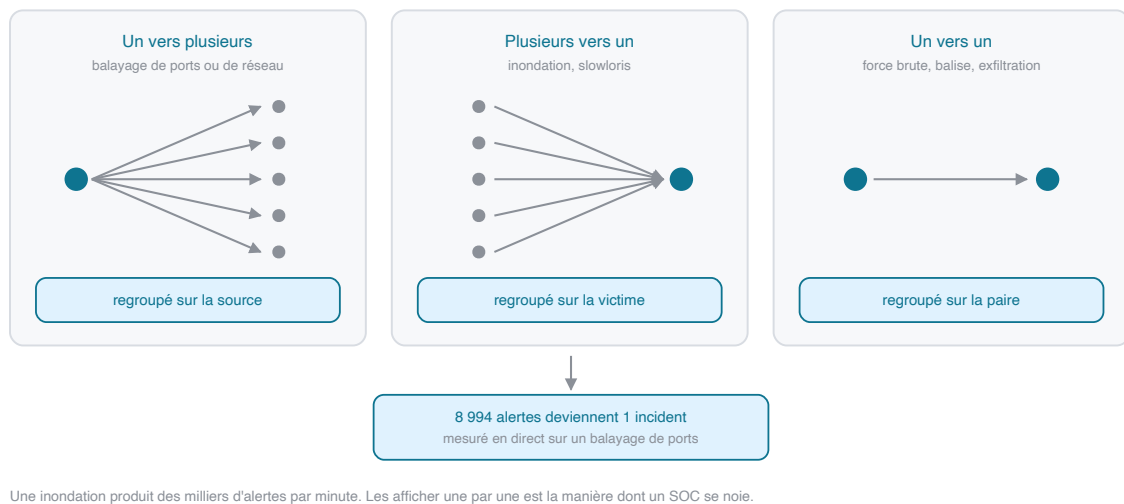


Figure 5 — La clé de regroupement suit la forme de l’attaque.

au-delà de 75, élevée au-delà de 58, moyenne au-delà de 38.

Un balayage de ports contre le serveur de base de données passe ainsi devant un balayage plus bruyant contre une imprimante — ce qui est le comportement attendu d’un outil de triage.

6.2 Cycle de vie d’un incident

Nouveau → pris en compte → en analyse → clos, avec un verdict parmi « vrai positif », « faux positif » et « trafic légitime ». Les verdicts sont enregistrés et constituent le signal de ré-entraînement ; toute action modifiante est journalisée avec son auteur.

Partie II — Mesures

7 Protocole

1. L'étage A n'est entraîné que sur du trafic **bénin** : il ne voit jamais d'attaque.
2. L'étage B est entraîné sur des captures étiquetées, plafonnées par classe.
3. La séparation entre entraînement et test se fait **par capture**, c'est-à-dire par « journée simulée », jamais par ligne. Des flux issus d'une même rafale d'attaque sont fortement corrélés : un découpage aléatoire placerait les frères d'un flux de test dans l'entraînement, et produirait les 99,9% illusoires dont ce type de projet est coutumier.
4. Le point de fonctionnement est obtenu en rejouant les captures de test **dans le moteur réel**, seuils adaptatifs compris, préchauffés sur une capture bénigne comme le serait une sonde venant d'être déployée — et non en seuillant une colonne de scores hors ligne.
5. Les faux positifs sont mesurés sur des captures **sans aucune attaque**, car la précision calculée sur une capture riche en attaques flatte n'importe quel détecteur.
6. La latence est comptée du début de la fenêtre d'attaque jusqu'à la première alerte, **délai d'export des flux inclus**.

Les graines d'entraînement sont 11 à 16, celles de test 901 à 903 : les deux ensembles sont disjoints.

8 Résultats globaux

Mesure	Attaques bruyantes	Attaques furtives
Rappel	99,56%	95,94%
Précision	99,89%	98,57%
F1	0,9973	0,9724
PR-AUC	0,9955	0,9373
ROC-AUC	0,9959	—
Latence médiane	2,0 s	6,0 s
Latence p95	120 s	120 s

Faux positifs : **54 incidents par heure** sur trafic sans attaque. Nommage de la famille : macro-F1 de 0,9997. Débit : environ 30 000 paquets par seconde en mono-processus ; décodage PCAP : 99 000 paquets par seconde.

9 Résultats par famille

Famille	MITRE	Bruyant	Furtif	n
Exfiltration de données	T1041	100,00%	100,00%	277
Déni de service par épuisement	T1499.003	100,00%	100,00%	584
Balayage réseau	T1018	99,93%	99,29%	10 524
Balayage de ports	T1046	99,93%	79,79%	5 378
Force brute SSH	T1110.001	99,69%	96,67%	2 549
Tunnel DNS	T1071.004	99,67%	96,87%	3 629
Déni de service volumétrique	T1498	99,50%	95,11%	19 538
Force brute applicative	T1110.004	99,48%	96,74%	2 681
Balise de commande et contrôle	T1071.001	48,62%	72,73%	109

9.1 Lecture des deux colonnes

Les variantes furtives sont les mêmes attaques ralenties d'un facteur quinze à soixante, avec rotation des adresses sources, variation des tailles de requête et gigue de 25% sur les balises.

Deux lignes méritent un commentaire.

Le balayage de ports chute de 99,9% à 79,8% en mode furtif. La rotation des sources dilue l'éventail de ports sur plusieurs adresses : c'est exactement la contre-mesure que prendrait un attaquant averti, et elle fonctionne partiellement.

La balise de commande et contrôle se détecte mieux en furtif (72,7%) qu'en bruyant (48,6%). Ce n'est pas une anomalie de mesure : la variante furtive a une période de 110 secondes, donc ses contacts s'étalent sur des fenêtres où l'hôte compromis est par ailleurs silencieux, ce qui les rend plus saillants.

9.2 Le cas de la balise

Le rappel de 48,6% est le résultat le plus instructif du tableau, et il est publié comme tel.

Une balise émet peu : quelques centaines d'octets toutes les dizaines de secondes, poignée de main complète, vers un port banal. Pris isolément, chacun de ses flux ressemble à une requête web ordinaire — c'est précisément l'effet recherché par un implant. Seule la **périodicité** la trahit, d'où la caractéristique `host_beacon_regularity`, et cette régularité n'apparaît qu'après plusieurs contacts : la fenêtre de soixante secondes doit en avoir vu assez.

Un rapport n'affichant que des scores supérieurs à 99% sur toutes les familles n'aurait pas mesuré un détecteur ; il aurait mesuré son propre générateur de trafic.

10 Précision : deux niveaux de lecture

La précision au niveau des **flux** atteint 99,89%. La précision au niveau des **incidents**, telle que jugée par les analystes dans l'interface, tourne autour de 76%.

Ce n'est pas une contradiction mais une conséquence de la corrélation : un faux positif ne fusionne jamais, il constitue son propre incident, alors qu'une attaque réelle fusionne des milliers d'alertes en une seule ligne. Le rapport entre les deux chiffres mesure donc exactement ce que la corrélation apporte.

Le tableau de bord affiche la valeur jugée par l'analyste, parce que c'est elle qui décrit la charge de travail réelle.

Partie III — Mise en œuvre

11 Interface et API

11.1 Les huit écrans

Vue d'ensemble	gravités, score et seuil sur 24 h, débit, familles, sources et cibles principales, précision observée
Flux temps réel	score en direct, débit, journal des incidents, compteurs du moteur
Incidents	table filtrable et triable, actions en masse
Détail d'incident	attribution, alertes fusionnées, ports, pairs, vérité terrain, guide de traitement, triage
Hôtes	profil par machine, seuil propre, incidents associés
Modèle	métriques, rappel par famille, matrice de confusion, seuils par hôte
Sources & rejeu	laboratoire, rejeu de PCAP, capture réelle
Réglages / Audit	point de fonctionnement, inventaire d'actifs, journal attribué

11.2 Transport et authentification

L'API expose une vingtaine de points d'entrée REST et un flux WebSocket. L'authentification se fait par jeton JWT d'une durée de douze heures ; trois rôles sont vérifiés *côté serveur* : lecteur (consultation), analyste (triage), administrateur (réglages et sources).

Les mots de passe sont hachés avec `hashlib.scrypt`, issu de la bibliothèque standard : une dépendance de moins, aucune dérive de version entre `passlib` et `bcrypt`, et une fonction mémoire-dure. La comparaison est à temps constant.

Le flux WebSocket s'authentifie par paramètre d'URL, l'API du navigateur ne permettant pas de poser un en-tête `Authorization` sur une connexion WebSocket.

11.3 Contre-pression

Chaque abonné au flux possède une file bornée et perd ses événements les plus anciens lorsqu'il ne suit pas. Perdre des images pour un onglet de navigateur vaut toujours mieux que bloquer l'ingestion des paquets pour tout le monde. Le même principe gouverne la capture réelle : le fil de capture ne bloque jamais, il compte les paquets perdus et l'interface les affiche.

12 Modèle de données

Table	Rôle
<code>incidents</code>	un groupe d'alertes corrélées : l'unité de travail de l'analyste
<code>alerts</code>	un flux ayant franchi le seuil de son hôte
<code>buckets</code>	agrégats par seconde, pour les graphiques sans parcourir les alertes
<code>hosts</code>	profil glissant par machine
<code>users</code>	comptes et rôles
<code>audit</code>	toute action modifiante, avec son auteur
<code>settings</code>	point de fonctionnement et inventaire d'actifs

Le schéma est décrit en SQLAlchemy et fonctionne indifféremment sur SQLite en mode WAL — ce qui permet à l'API de lire pendant que la tâche d'ingestion écrit — ou sur PostgreSQL, par simple changement de l'URL de connexion. Rien dans le schéma n'est propre à SQLite.

Les alertes sont ajoutées à un incident jusqu'à un plafond : conserver la dix-millième requête SYN identique d'une inondation n'a aucune valeur d'investigation et ferait croître la base sans limite. Le compteur d'occurrences, lui, continue de raconter toute l'histoire.

13 Les trois sources de trafic

13.1 Réseau de laboratoire

Un générateur produit des **paquets** — et non des flux déjà agrégés — étiquetés par famille. C’est ce qui permet d’exercer la chaîne de bout en bout, assemblage compris, et de disposer d’une vérité terrain.

Le trafic bénin mélange navigation web interne et externe, requêtes DNS, interrogations de base de données, partage de fichiers, sessions SSH interactives, diffusion vidéo, NTP et ICMP, ainsi qu’une part de trafic légitimement inhabituel — sans quoi le modèle apprendrait qu’inhabituel équivaut à malveillant.

Le trafic est construit par tranches de six cents secondes simulées. La version initiale planifiait toutes les sessions d’une période à l’avance : demander vingt-quatre heures revenait à construire près d’un demi-million de générateurs avant d’émettre le premier paquet, et le démarrage de l’API se bloquait.

13.2 Rejeu de capture

Le décodage utilise un lecteur direct écrit en **struct** pour le cas courant — pcap classique, Ethernet et IPv4 — avec repli sur scapy pour les types de liaison exotiques. Scapy plafonnait à 3 200 paquets par seconde, ce qui imposait cinq minutes d’attente avant le premier flux sur une capture d’un million de paquets ; le lecteur direct atteint **99 000 paquets par seconde**.

L’écriture suit le même chemin : un écrivain direct, avec une longueur de capture tronquée qui conserve néanmoins l’intégralité de l’échantillon de charge utile, de sorte que les compteurs d’octets et l’entropie survivent exactement à l’aller-retour.

Fidélité vérifiée : les caractéristiques structurelles — 5-uplet, paquets, octets, entropie, éventail de ports — sont identiques à 100% entre la génération en direct et le rejeu du fichier. Seuls les horodatages subissent la quantification à la microseconde propre au format pcap.

13.3 Capture réelle

Par scapy, sur un fil séparé, avec une file bornée. Elle exige les privilèges administrateur pour accéder aux périphériques BPF. **Ce chemin n’a pas été exercé** : le code existe et sa gestion de contre-pression est testée, mais aucune capture réelle n’a été effectuée.

14 Inventaire des tests

Fichier	Nb	Ce qui est vérifié
test_core.py	17	assemblage, entropie, délais d’export, équivalence de l’expiration optimisée avec le parcours naïf, fenêtres glissantes, éventail de scan, inondation usurpée
test_detection.py	19	calibration contre une loi normale, monotonie, queue de distribution, seuil par hôte, garde-fou de la grenouille ébouillantée, clés de corrélation, criticité des actifs, cascade
test_api.py	21	authentification, jeton forgé, trois rôles, triage, notes, audit, validation des réglages, traversée de répertoire
test_sources.py	30	ordre temporel du flux, reproductibilité, chaque famille produit des flux, variantes furtives plus discrètes, aller-retour PCAP octet pour octet, contre-pression
test_model.py	7	persistance, refus d’un jeu de caractéristiques différent, refus d’un format ancien, bornes des scores, attribution
Total	94	

Deux tests méritent d’être signalés. Le premier prouve que l’expiration optimisée produit exactement les mêmes flux que l’implémentation naïve, sur quatre minutes de trafic mixte. Le second exerce le

vrai chemin d'écriture en base : un bug y vivait — SQLAlchemy applique les valeurs par défaut à l'INSERT, si bien qu'une ligne fraîchement construite contenait encore `None` et que la première incrémentation tuait la tâche d'ingestion — et aucun test unitaire sur l'écrivain ne l'aurait vu.

Les schémas sont également vérifiés : leur générateur les régénère et refuse qu'un texte déborde de sa toile, ce qui empêche un schéma de vieillir en silence pendant que le projet avance.

15 Déploiement

```
git clone https://github.com/aziz3r/aegis-mini-soc.git && cd aegis-mini-soc
./scripts/demo.sh
```

Le script installe ce qui manque, entraîne le modèle s'il est absent (environ six minutes, une seule fois), crée la base et les comptes, puis démarre l'API avec le réseau de laboratoire déjà en marche.

Aucune dépendance système n'est requise : ni Docker, ni serveur de base de données, ni privilèges administrateur — sauf pour la capture d'une interface réelle, qui en exige par nature. Si le port est occupé, le script le signale et propose une alternative plutôt que d'échouer sur une erreur de liaison.

16 Limites et orientations

Non traité	Le trafic d'évaluation est synthétique. La chaîne de détection est réelle et identique à celle qui tourne en production, mais un réseau réel est plus désordonné et produira davantage de faux positifs.
Partiel	Les familles sont séparables par construction, ce qui explique le macro-F1 très élevé de l'étage B sur les variantes bruyantes. La colonne furtive est la mesure honnête.
Non traité	La capture d'une interface réelle n'a jamais été exercée, faute de privilèges administrateur.
Partiel	La détection est par flux, non par paquet : la latence inclut le délai d'export.
Non traité	Sonde unique, sans agrégation multi-capteurs ; pas d'inspection applicative.

Les suites, par ordre de valeur : rejeu validé sur CIC-IDS2017 avec ses étiquettes officielles, ce qui permettrait la comparaison avec la littérature ; ré-entraînement sur les verdicts d'analystes, déjà enregistrés mais non exploités ; suivi de dérive entre la distribution d'entraînement et le trafic du jour ; réponse automatique en mode simulation par défaut ; agrégation de plusieurs sondes.